

LABSHEET-2**NITIN PANWAR****CU240251575****SEC- B****ROLL NO- 39**

2. Import the required Python libraries (Pandas NumPy, Matplotlib, and Seaborn) and load the cleaned dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.set_style("whitegrid")
```

```
df = sns.load_dataset("titanic")
```

```
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	female	38.0	1	0	71.2833	C	First		
2	1	female	26.0	0	0	7.9250	S	Third		
3	1	female	35.0	1	0	53.1000	S	First	4	0
4	0	male	35.0	0	0	8.0500	S	Third		

	who	adult_male	deck	embark_town	alive	alone	0
man		True	NaN	Southampton	no	False	
1	woman		False	C	Cherbourg	yes	False
2	woman		False	NaN	Southampton	yes	True
3	woman		False	C	Southampton	yes	False
4	man		True	NaN	Southampton	no	True

3. Explore the dataset by examining its dimensions, data types, column names, and summary statistics using functions such as info(), describe(), and value_counts().

```
print("Shape of dataset:", df.shape)
print("\nColumn names:\n", df.columns.tolist())
```

Shape of dataset: (891, 15)

Column names:

['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'cla

```
print("\nData types & non-null info:")
df.info()
```

Data types & non-null info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	deck	203 non-null	category
12	embark_town	889 non-null	object
13	alive	891 non-null	object
14	alone	891 non-null	bool

dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

```
print("\nSummary statistics (numerical columns):")
print(df.describe())
```

Summary statistics (numerical columns):

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
print("\nSummary statistics (categorical columns):")
print(df.describe(include="object"))
```

Summary statistics (categorical columns):

	sex	embarked	who	embark_town	alive
count	891	889	891	889	891
unique	2	3	3	3	2
top	male	S	man	Southampton	no
freq	577	644	537	644	549

```
print("\nMissing values per column:")
print(df.isnull().sum())
```

Missing values per column:

survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0

dtype: int64

```
print("\nValue counts for 'class':")
print(df["class"].value_counts())
```

Value counts for 'class':

class	
Third	491
First	216
Second	184

Name: count, dtype: int64

```
print("\nValue counts for 'sex':")
print(df["sex"].value_counts())
```

Value counts for 'sex':

sex	
male	577
female	314

Name: count, dtype: int64

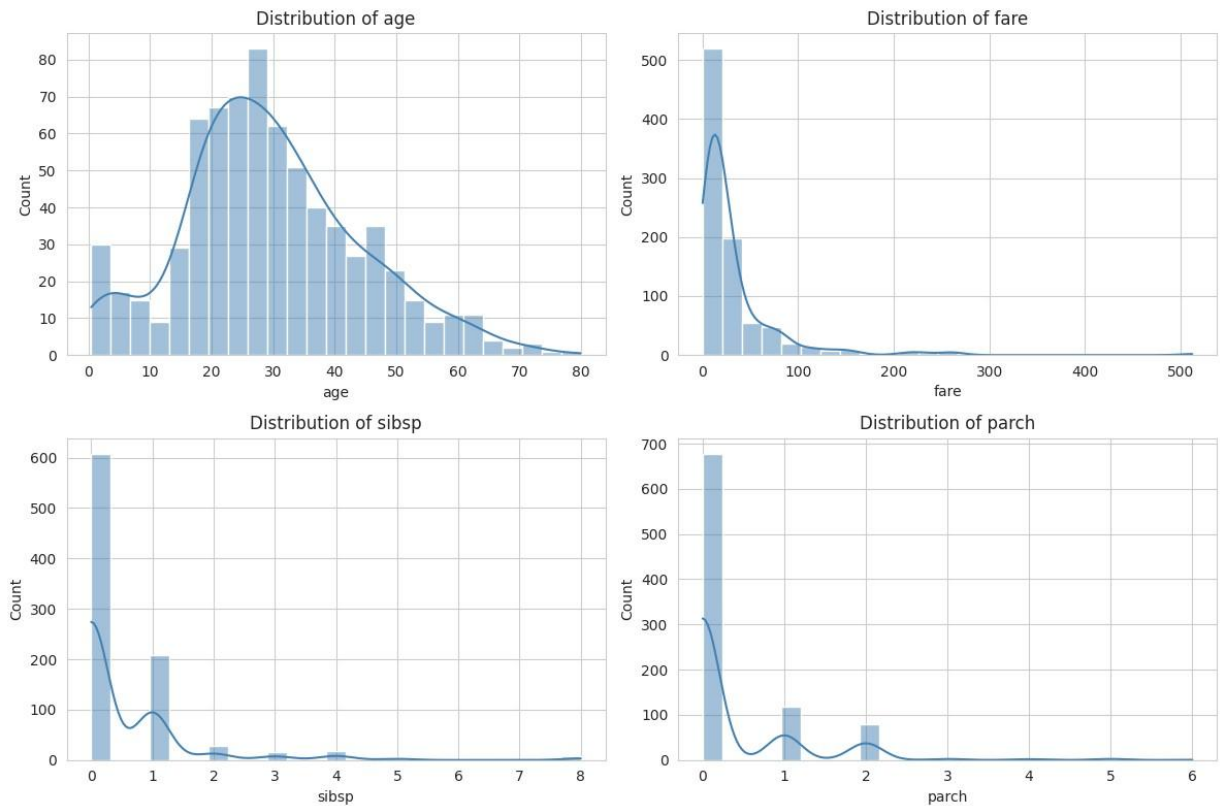
4. Analyze the distribution of numerical features using histograms and densityplots to understand central tendency and data spread.

```

numeric_cols = ["age", "fare", "sibsp", "parch"]

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, col in zip(axes.flatten(), numeric_cols):
    sns.histplot(df[col].dropna(), kde=True, bins=25, ax=ax, color="steelblue")
    ax.set_title(f"Distribution of {col}")
plt.tight_layout()
plt.show()

```



5. Visualize categorical variables using bar charts and count plots to identify the most frequent categories.

```

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
sns.countplot(data=df, x="sex", hue="sex", palette="Set2", legend=False,
ax=axes[0].set_title("Passenger Count by Sex"))

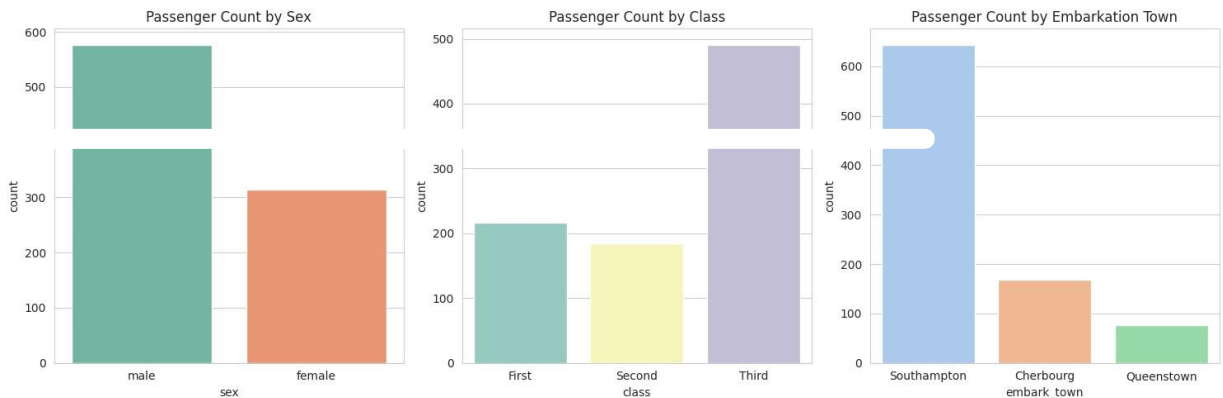
```

```
sns.countplot(data=df, x="class", hue="class", palette="Set3", legend=False, axes[1].set_title("Passenger Count by Class"))
```

```
sns.countplot(data=df, x="embark_town", hue="embark_town", palette="pastel", le axes[2].set_title("Passenger Count by Embarkation Town"))
```

```
plt.tight_layout()
```

```
plt.show()
```



6. Generate a correlation matrix and heatmap to identify relationships between numerical variables and detect highly correlated features.

```
corr = df.select_dtypes(include="number").corr()
print("Correlation matrix:\n", corr)
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap of Numerical Features", fontsize=14, fontweight=
plt.show())
```

Correlation matrix:

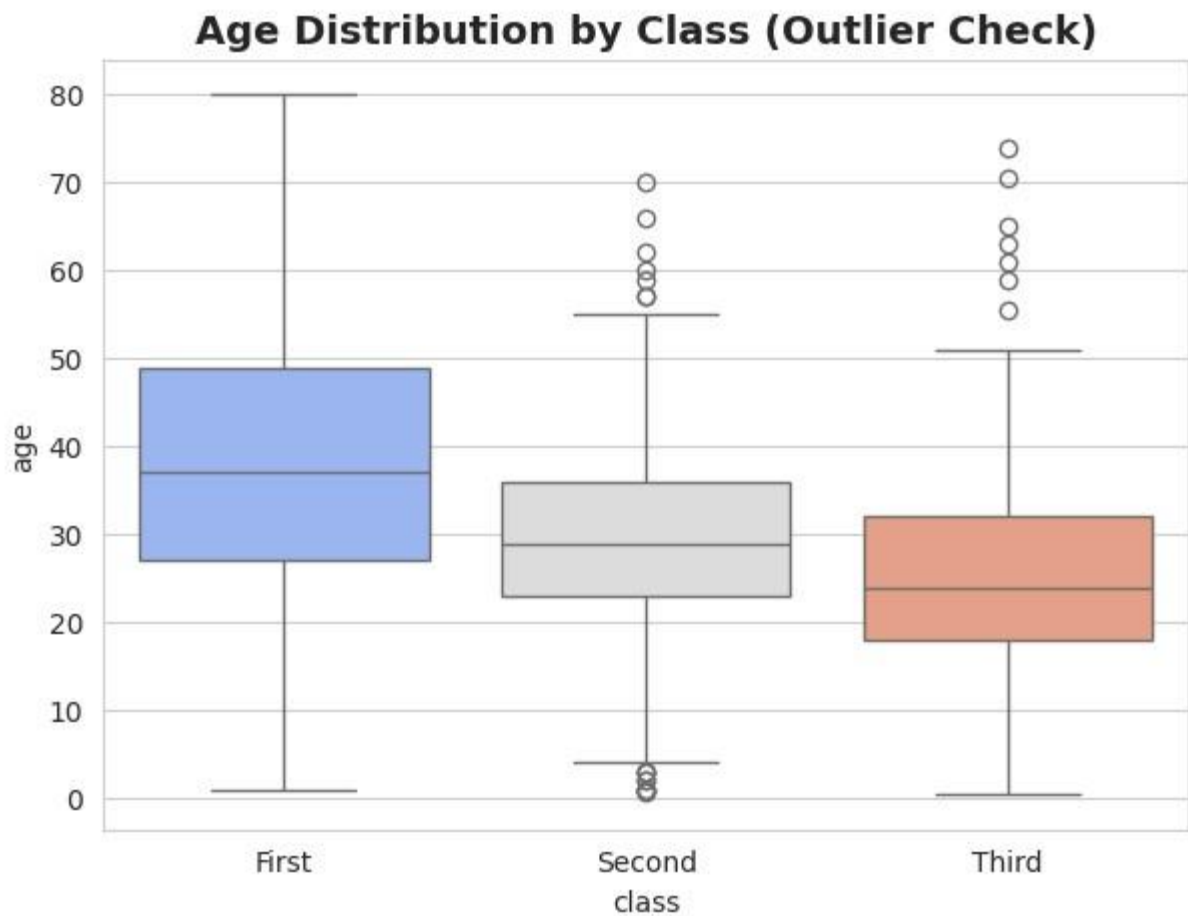
	survived	pclass	age	sibsp	parch	fare
survived	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
pclass	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
age	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
sibsp	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
parch	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
fare	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

Correlation Heatmap of Numerical Features

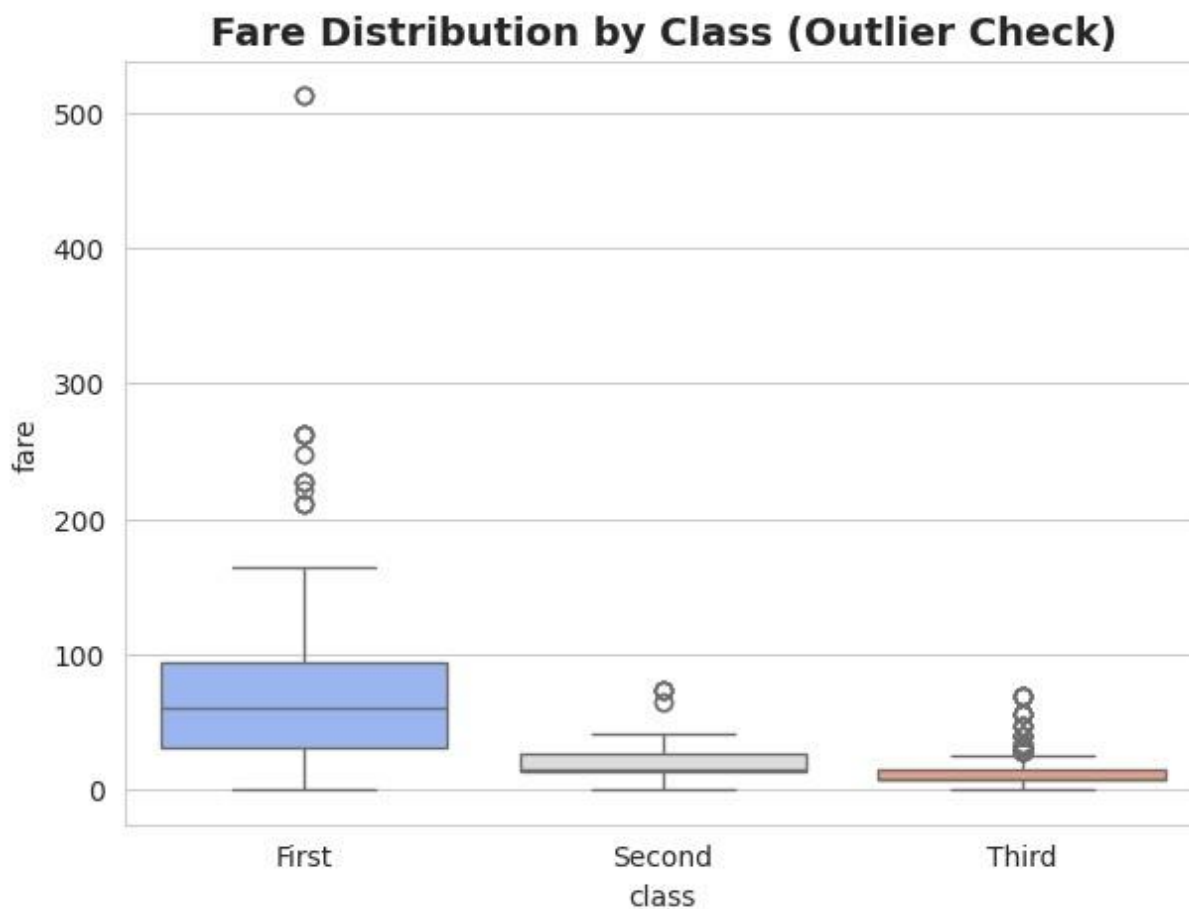


7. Create box plots and scatter plots to identify outliers and study relationships between different variables.

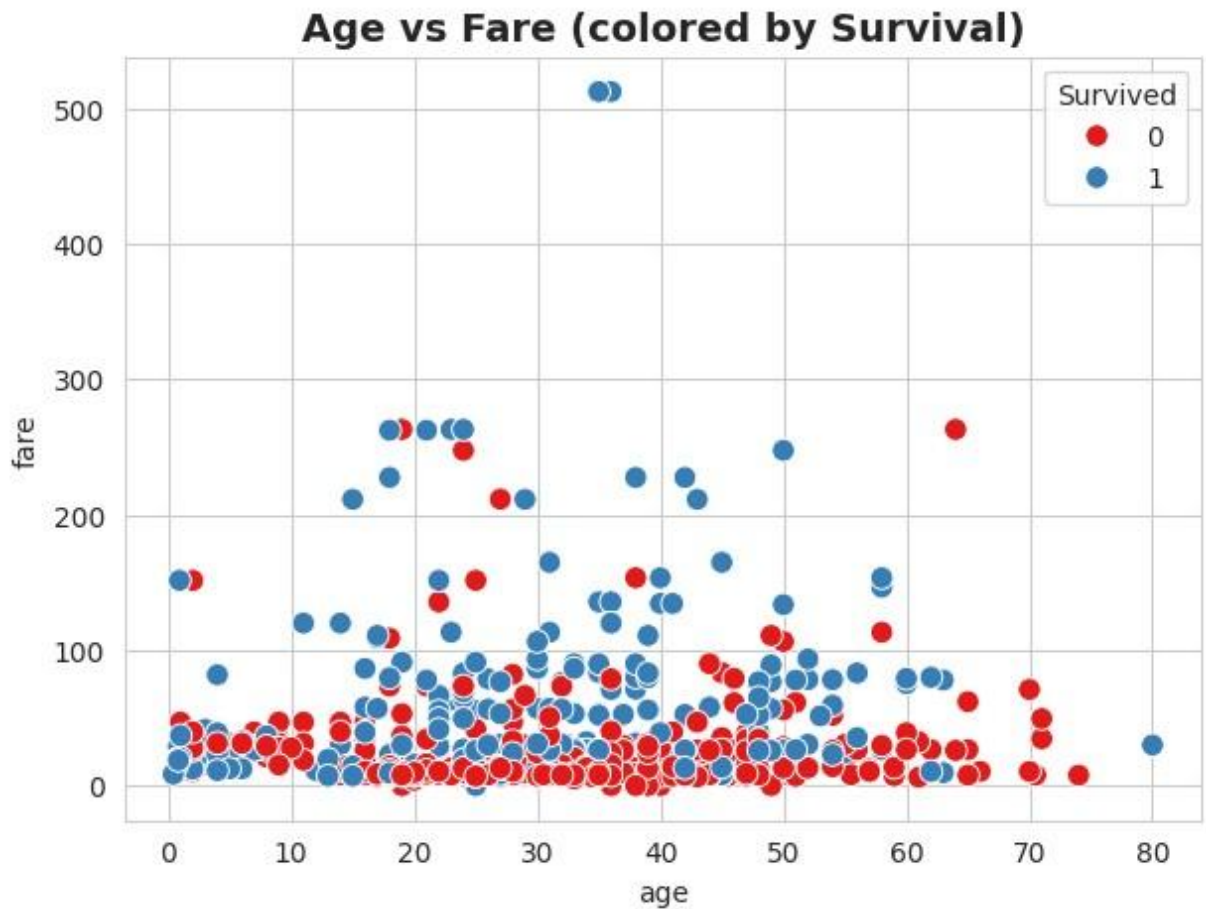
```
plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x="class", y="age", hue="class", palette="coolwarm", legend=True)
plt.title("Age Distribution by Class (Outlier Check)", fontsize=14, fontweight="bold")
plt.show()
```



```
plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x="class", y="fare", hue="class", palette="coolwarm", legend=True)
plt.title("Fare Distribution by Class (Outlier Check)", fontsize=14, fontweight="bold")
plt.show()
```

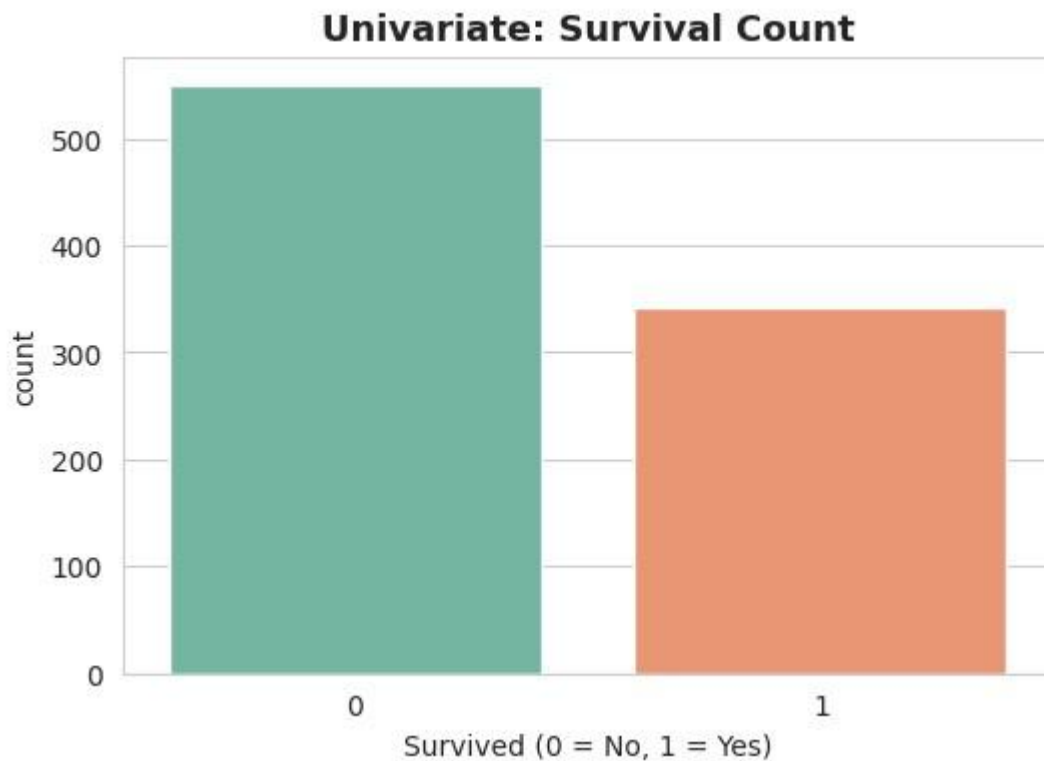


```
plt.figure(figsize=(7, 5))
sns.scatterplot(data=df, x="age", y="fare", hue="survived", palette="Set1", s=7)
plt.title("Age vs Fare (colored by Survival)", fontsize=14, fontweight="bold")
plt.legend(title="Survived")
plt.show()
```

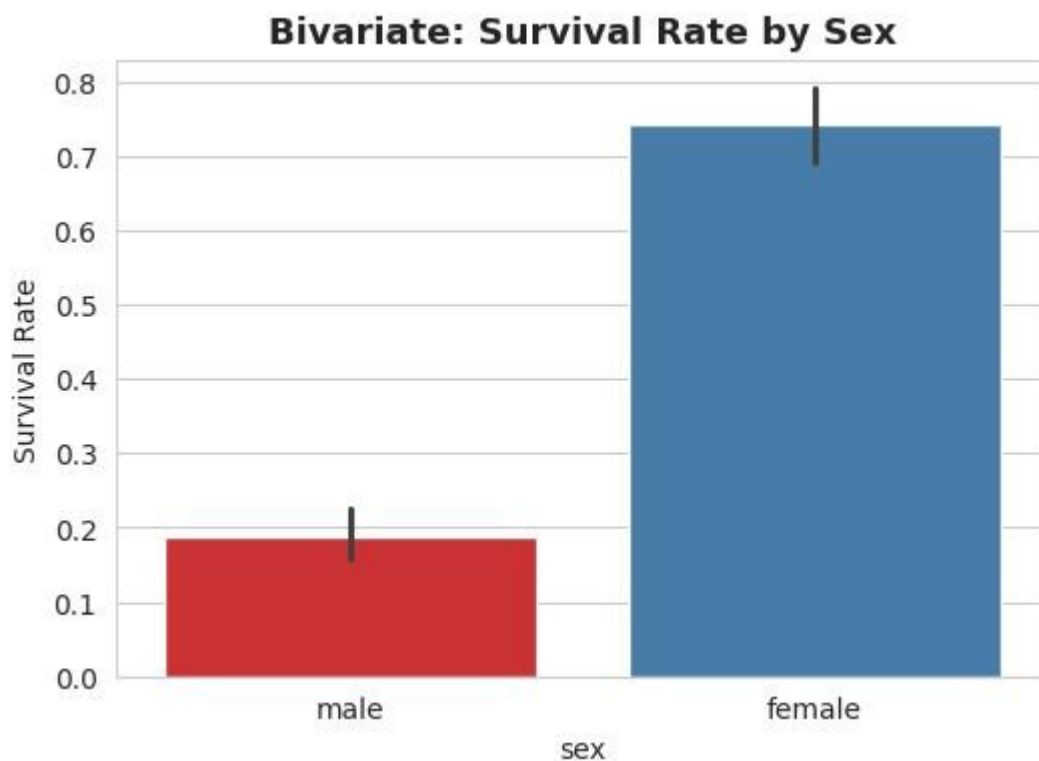


8. Perform univariate, bivariate, and multivariate analysis to uncover trends and business patterns within the dataset.

```
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="survived", hue="survived", palette="Set2", legend=False)
plt.title("Univariate: Survival Count", fontsize=13, fontweight="bold")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.show()
```



```
plt.figure(figsize=(6, 4))
sns.barplot(data=df, x="sex", y="survived", hue="sex", palette="Set1", legend=F
plt.title("Bivariate: Survival Rate by Sex", fontsize=13, fontweight="bold")
plt.ylabel("Survival Rate")
plt.show()
```



```
plt.figure(figsize=(7, 5))
```